UTEC
Universidad Tecnológica

Patrón Strategy

Patrón Strategy: Encapsulamiento de Algoritmos Intercambiables

Descripción General

El patrón **Strategy** (o Estrategia) es un patrón de diseño de comportamiento que permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. Este patrón permite que el algoritmo varíe independientemente de los clientes que lo utilizan, facilitando la extensión y modificación del comportamiento sin cambiar el código del cliente.

Componentes Principales

1. **Estrategia (Strategy)**: Es una interfaz que define un conjunto de métodos para los algoritmos que serán implementados por las estrategias concretas.
2. **Estrategias Concretas (Concrete Strategies)**: Son las implementaciones concretas de la interfaz de estrategia. Cada clase implementa un algoritmo específico.
3. **Contexto (Context)**: Es la clase que utiliza una estrategia concreta para realizar un comportamiento. El contexto no implementa los algoritmos por sí mismo, sino que delega su ejecución a la estrategia seleccionada.

Ventajas

- **Flexibilidad**: Permite cambiar el algoritmo usado en tiempo de ejecución sin modificar el código cliente.
- **Principio de Abierto/Cerrado**: Facilita la adición de nuevas estrategias sin alterar las existentes.
- **Desacoplamiento**: Separa el comportamiento de una clase del algoritmo específico que se utiliza.

Desventajas

- **Complejidad adicional**: Aumenta el número de clases en el sistema, lo que puede hacer que el diseño sea más complejo.
- **Mantenimiento**: Si hay muchas estrategias, puede ser difícil mantenerlas y entender cómo interactúan entre sí.

UTEC
Universidad Tecnológica

Ejemplo en Java

Supongamos que tenemos una aplicación que procesa pagos y queremos implementar diferentes métodos de pago como tarjetas de crédito, PayPal y transferencias bancarias. Usaremos el patrón Strategy para encapsular los diferentes algoritmos de pago.

```
// Interfaz Strategy
interface MetodoPago {
    void procesarPago(double monto);
}

// Estrategia Concreta: Pago con tarjeta de crédito
class PagoTarjetaCredito implements MetodoPago {
    @Override
    public void procesarPago(double monto) {
        System.out.println("Procesando pago de $" + monto + " con
tarjeta de crédito.");
    }
}

// Estrategia Concreta: Pago con PayPal
class PagoPayPal implements MetodoPago {
    @Override
    public void procesarPago(double monto) {
        System.out.println("Procesando pago de $" + monto + " con
PayPal.");
    }
}

// Estrategia Concreta: Pago con transferencia bancaria
class PagoTransferenciaBancaria implements MetodoPago {
    @Override
    public void procesarPago(double monto) {
        System.out.println("Procesando pago de $" + monto + " con
transferencia bancaria.");
    }
}

// Contexto que utiliza las estrategias de pago
class ProcesadorPago {
    private MetodoPago metodoPago;
```

UTEC
Universidad Tecnológica

```
    // Se permite cambiar la estrategia en tiempo de ejecución
    public void setMetodoPago(MetodoPago metodoPago) {
        this.metodoPago = metodoPago;
    }

    public void procesar(double monto) {
        metodoPago.procesarPago(monto);
    }
}

// Ejemplo de uso
public class StrategyPatternExample {
    public static void main(String[] args) {
        ProcesadorPago procesador = new ProcesadorPago();

        // Usando la estrategia de pago con tarjeta de crédito
        procesador.setMetodoPago(new PagoTarjetaCredito());
        procesador.procesar(100.0);

        // Cambiando a la estrategia de pago con PayPal
        procesador.setMetodoPago(new PagoPayPal());
        procesador.procesar(150.0);

        // Cambiando a la estrategia de pago con transferencia bancaria
        procesador.setMetodoPago(new PagoTransferenciaBancaria());
        procesador.procesar(200.0);
    }
}
```

Explicación del Ejemplo

- **MetodoPago**: Es la interfaz de la estrategia que define el método `procesarPago()` para procesar los pagos.
- **PagoTarjetaCredito, PagoPayPal, PagoTransferenciaBancaria**: Son las estrategias concretas que implementan el procesamiento de pagos con diferentes métodos.
- **ProcesadorPago**: Es el contexto que utiliza una estrategia de pago para procesar los pagos. Permite cambiar la estrategia en tiempo de ejecución, facilitando la flexibilidad del sistema.

UTEC
Universidad Tecnológica

Otros Usos Comunes del Patrón Strategy

- **Compresión de Archivos**: Donde diferentes estrategias de compresión (ZIP, RAR, GZIP) pueden ser intercambiadas fácilmente.
- **Cálculo de Impuestos**: Donde diferentes estrategias de cálculo de impuestos pueden ser aplicadas según la región o tipo de producto.
- **Algoritmos de Búsqueda y Ordenación**: Donde diferentes estrategias de búsqueda u ordenación pueden ser aplicadas dependiendo de la situación.

Conclusión

El patrón Strategy es ideal para situaciones donde un comportamiento debe variar en función de alguna condición, permitiendo que los algoritmos sean seleccionados o intercambiados en tiempo de ejecución. Esto mejora la flexibilidad y extensibilidad del código, permitiendo a los desarrolladores agregar nuevas estrategias sin modificar las clases existentes, alineándose con los principios de diseño SOLID, particularmente con el principio de abierto/cerrado.